

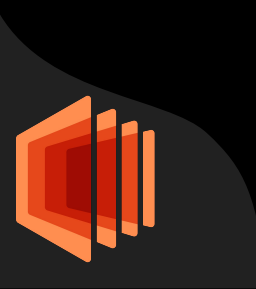


Introduction to Vitess sharding framework for MySQL

Matthias Crauwels

Enterprise Customer Engineer, PlanetScale

[@mcrauwel](#)



// Introduction

Who am I?

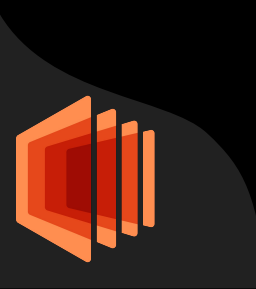
- Living in Ghent, Belgium 🇧🇪
- Bachelor Computer Science
- ~30 years Linux user / admin
- ~20 years PHP developer
- ~15 years MySQL DBA
- 4th year at PlanetScale
- Enterprise Customer Engineer
- Father of Leander
- MySQL Rockstar 2025



About PlanetScale

PlanetScale is the world's most scalable and reliable OLTP database. Built on Vitess, our cloud platform is trusted by some of the world's largest brands. We offer a range of deployment options including multi-tenant cloud, single-tenant, or bring your own cloud account with PlanetScale Managed.

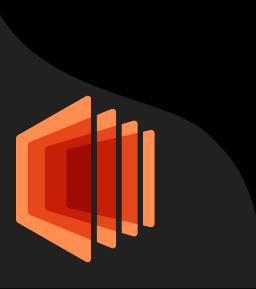
PlanetScale was founded in 2018 by the co-creators and maintainers of the Vitess open source project. We are a globally-distributed company that values high performance, accountability, and integrity.



About Vitess

PlanetScale is powered by Vitess, the open-source database technology that was invented at YouTube in 2010 to solve the scaling issues they faced with their massive MySQL database.

Vitess went on to become open source as a CNCF project and continues to scale massive companies like PlanetScale, Slack, GitHub, and more.



Vitess serves **millions of QPS** in production

 **slack**

 New Relic®

 Square

Flipkart

HubSpot

peak

 Pinterest

 *shopify*

 nozzle

 weave

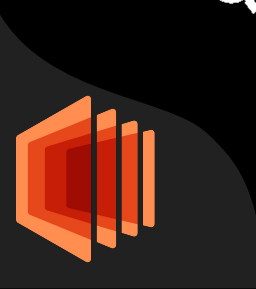
GitHub

JD.京东
.COM

Quiz of Kings

 stitchlabs

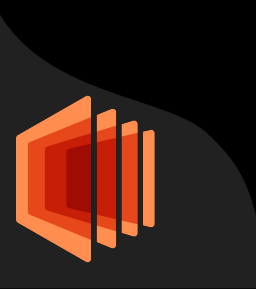
 PlanetScale



// Introduction

Agenda

- Architecture
- MySQL Compatibility
- VReplication
- Online Schema Changes
- Automatic failovers



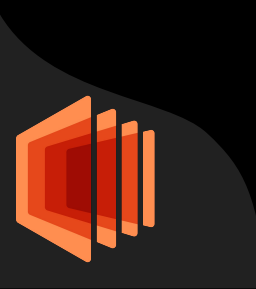
Architecture



// Architecture

What is Vitess?

- Cloud Native Database
- Massively Scalable
- Highly Available
- MySQL Compatible



// Architecture

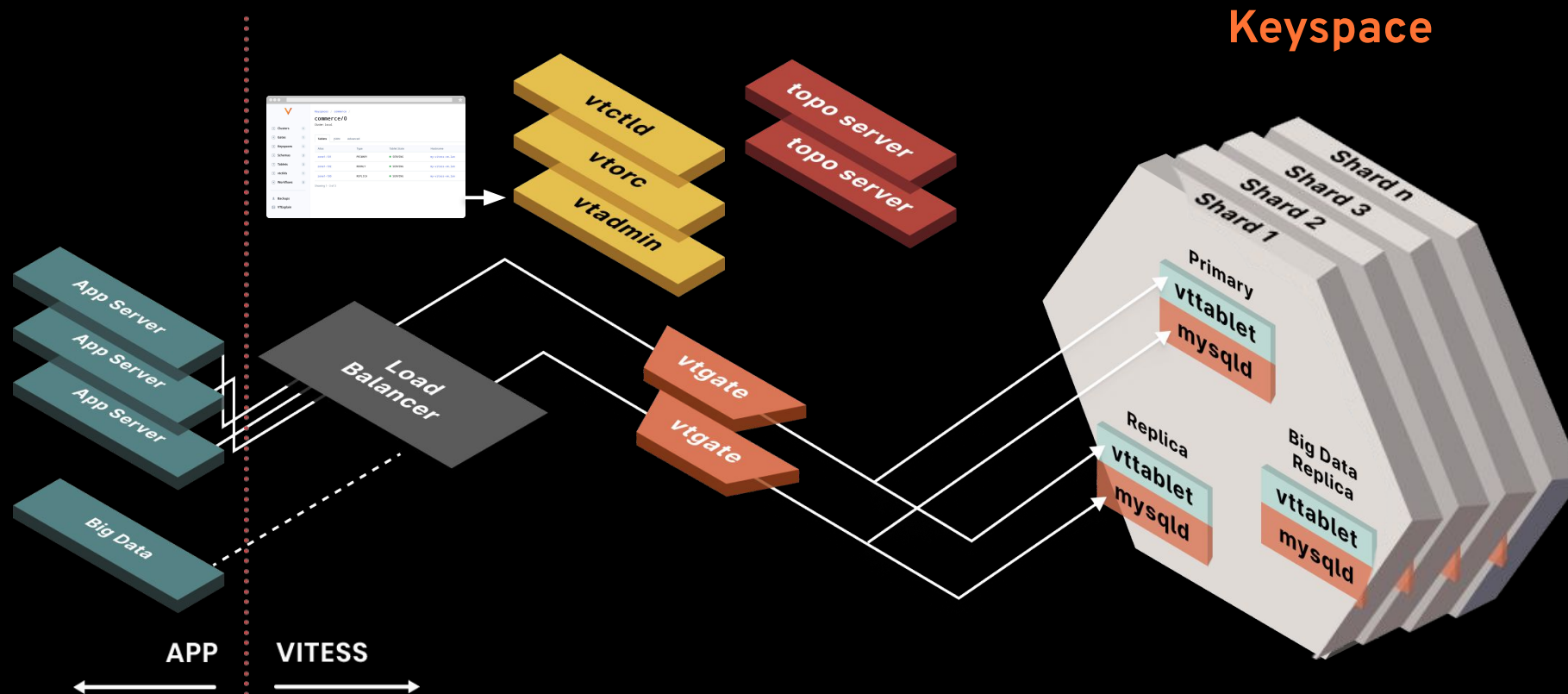
Concepts

- Keyspace
- Shard
- Topology server



// Architecture

Vitess Architecture



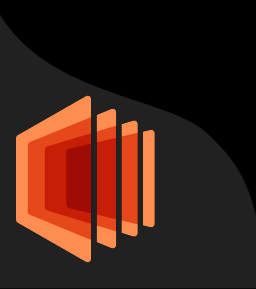
MySQL Compatibility



// MySQL Compatibility

Vitess provides an illusion

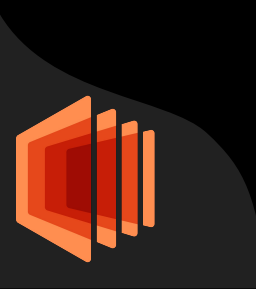
- A single database (server)
- MySQL 8.0 and above
- A single, dedicated connection



// MySQL Compatibility

Vitess needs to deal with

- Database frameworks
- ORMs
- Legacy code
- Third Party Applications



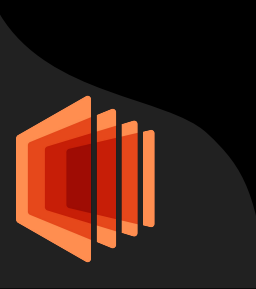
VReplication



// VReplication

Use cases

- Migration
 - Import data into Vitess
- (Re)sharding
 - Move data around
- Materialisation
 - Improve query performance
 - Materialise unsharded data on each shard



Import workflow

- Create an “external” **keyspace** with a **vttablet** pointing to an existing MySQL server.
- Create a **Vitess keyspace**
- Use the **MoveTables** command to create a **VReplication Workflow** to move data into Vitess
- Confirm that all data was migrated (**VDiff**)
- **Switch read traffic** to go to Vitess
- **Switch write traffic** to go to Vitess
- Full support for **rollback** in case of issues



Regular MySQL as source

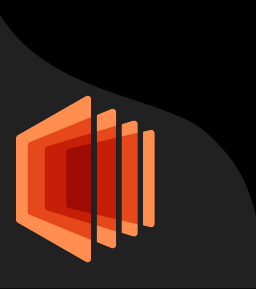
- Binary logs are required
 - When copying from a replica GTIDs and `log_replica_updates` are required
 - `binary log format = ROW`
 - `binary log image = FULL`
- 2 phases
 - Copy phase
 - Long running select - `vreplication_copy_phase_duration` (default 1h)
 - Throttling
 - History List Length - `vreplication_copy_phase_max_innodb_history_list_length` (default 1 million)
 - Replication lag (source) - `vreplication_copy_phase_max_mysql_replication_lag` (default 12h)
 - Catch Up phase
 - After each chunk of the copy phase
 - Apply binary log event for the already copied rows
 - Avoids long catch up phase on large tables



// VReplication

Aurora MySQL as source

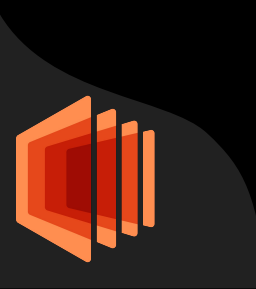
- No binary logs on Aurora readers
 - Set up a replica Aurora cluster doing binary log replication from the main cluster
 - Use the primary of this cluster as "replica" tablet
- Server UUID (GTID) is the same for all nodes on the source cluster
 - Stream survives a Aurora failover
- When you don't have GTID (enabling requires an Aurora reboot) repoint vreplication to the primary cluster before cutover
 - `STOP REPLICATION` on replica cluster
 - `SHOW BINARY LOG STATUS` on replica cluster
 - ensure vreplication catches up (File/Position from the previous output)
 - `SHOW REPLICATION STATUS` on replica-cluster
 - fetch `Source_Log_File` and `Executed_Source_Log_Pos`
 - stop vreplication workflow
 - `UPDATE _vt.replication SET pos = '...', tablet_types = 'PRIMARY' WHERE id = ...` on target cluster(s)
 - start vreplication workflow



// VReplication

Sharded MySQL / Vitess as source

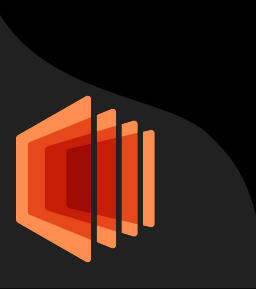
- Shard by shard migration
 - Sharded source keyspace
 - Sharded target keyspace
 - Same shard definitions for both source and target
 - Requires vtgate flag `--enable-partial-keyspace-migration`
 - MoveTables stream per shard
 - Ability to copy in segments not to overload uplink
 - Ability to switch-reads and switch-writes individually
 - No "big bang" migration
 - Fully reversible migrations
 - `ShardRoutingRules`
 - to be able to switch traffic on a per-shard basis
 - Executed this with cluster up to 256 shards (not the hard limit)
 - Some manual cleanup at the end



Multi-tenant migrations

- Multiple (identical) schema spread over 1 or more MySQL cluster(s)
 - Unique tenant identifier required in all tables (immutable)
 - Each source tenant is it's own external keyspace
 - Queries should include `WHERE <field_tenant_id> = ...` in all `SELECT/UPDATE/DELETE` statements
 - Target schema
 - Add `<field_tenant_id>` to PRIMARY KEY (required for uniqueness)
 - Add `<field_tenant_id>` to secondary indexes (as first field to use for filtering)
 - HowTo
 - VSchema add `multi_tenant_spec` to the top-level JSON

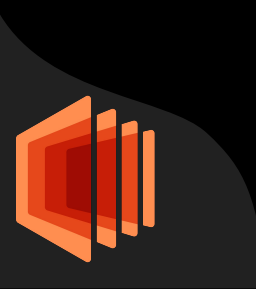
```
"multi_tenant_spec": {
    "tenant_id_column_name": "<field_tenant_id>",
    "tenant_id_column_type": "INT64"
},
```
 - MoveTables specify `--tenant_id <value>` during Create
 - Merges 100s or 1000s of schema's into one (sharded) keyspace



// VReplication

Multi-tenant migrations (2)

- `KeyspaceRoutingRules`
 - Introduced for multi-tenant migrations
 - All keyspaces have the same tables
 - Works very similar to normal `RoutingRules` and `ShardRoutingRules`
- When sharding the Vitess cluster on `<field_tenant_id>`
 - option to specify `--shards` option to `MoveTables Create`



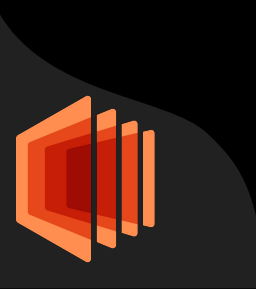
Online Schema Changes



// Online Schema Changes

ALTER TABLE issues

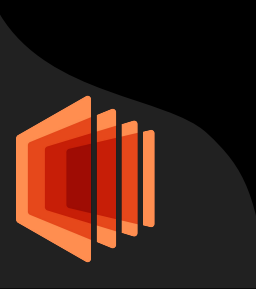
- Blocking
- Resource greedy
- Not auditable
- Not interruptible



// Online Schema Changes

ALTER TABLE Workarounds

- gh-ost, developed by GitHub
- pt-online-schema-change, part of Percona Toolkit
- External tools
- Requires a lot of operational knowledge of the environment
- Takes control out of the developer's hands



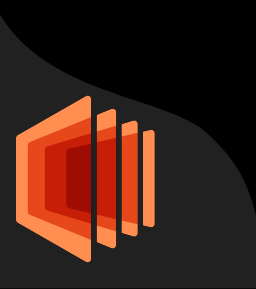
// Online Schema Changes

Based on VReplication

```
mysql> SET @@ddl_strategy = 'online';  
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> ALTER TABLE table1 CHANGE id id bigint unsigned;  
+-----+  
| uuid                                     |  
+-----+  
| 4d8246f2_9801_11ed_a6ae_c87f5403e983 |  
+-----+  
1 row in set (0.02 sec)
```

- Grab a coffee!

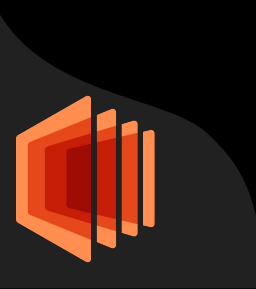


// Online Schema Changes

Based on VReplication

```
mysql> SHOW vitess_migrations LIKE '4d8246f2_9801_11ed_a6ae_c87f5403e983'\G

***** 1. row *****
...
  migration_uuid: 4d8246f2_9801_11ed_a6ae_c87f5403e983
  added_timestamp: 2023-01-19 14:58:08
  completed_timestamp: 2023-01-19 14:58:18
  migration_status: complete
  migration_statement: alter table table1 change column id id bigint unsigned
                    strategy: online
...
1 row in set (0.00 sec)
```



// Online Schema Changes

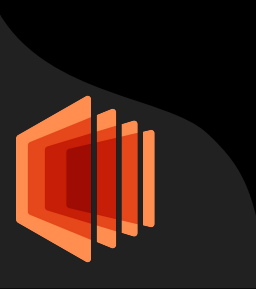
But there is more...

- Migrations become fully reversible
- Because the VReplication stream could easily be reversed, the old table is kept up-to-date.
- Now a revert operation is as simple as

```
mysql> REVERT vitess_migration '4d8246f2_9801_11ed_a6ae_c87f5403e983';
```

```
+-----+  
| uuid                                     |  
+-----+  
| 20f5337f_9826_11ed_a6ae_c87f5403e983 |  
+-----+
```

```
1 row in set (0.04 sec)
```



// Online Schema Changes

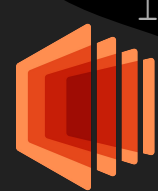
Declarative migrations

- No need to write `ALTER` statements anymore, just resubmit the `CREATE TABLE` statement and Vitess will figure out the difference and run the migration...

```
mysql> SET @@ddl_strategy = 'online -declarative';  
Query OK, 0 rows affected (0.00 sec)
```

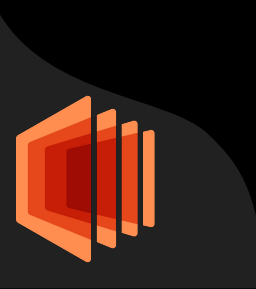
```
mysql> CREATE TABLE `table1` (  
->   `id` bigint unsigned NOT NULL,  
->   `data` varchar(512) DEFAULT NULL,  
->   PRIMARY KEY (`id`)  
-> ) ENGINE=InnoDB;
```

```
+-----+  
| uuid                                     |  
+-----+  
| c423f39b_982c_11ed_a6ae_c87f5403e983 |  
+-----+  
1 row in set (0.02 sec)
```



Conclusion

- Schema changes are being put back into the hands of the developers!
- Easy to run
- Easy to revert
- Due to the robustness of VReplication, it can even survive a primary failover or other type of outage



Automatic failovers



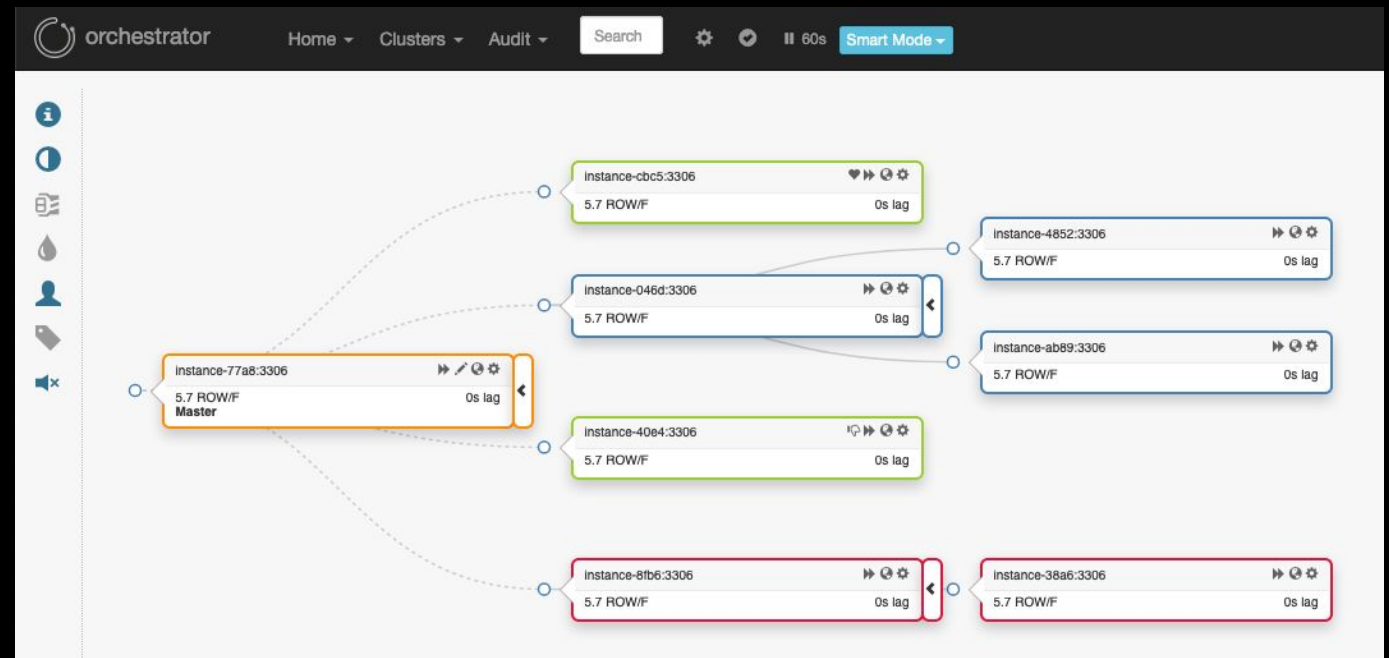
// Automatic failovers

Orchestrator

Orchestrator is open-source software commonly used as a MySQL Topology management tool. It's purpose is to observe MySQL replication topologies and potentially fix these topologies when failures are observed.

Core functionalities:

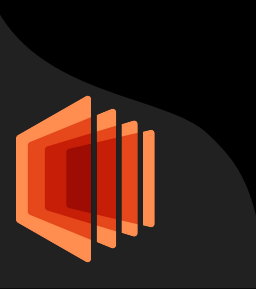
- Discover
- Visualize
- Monitor
- Refactor



// Automatic failovers

VTOrc

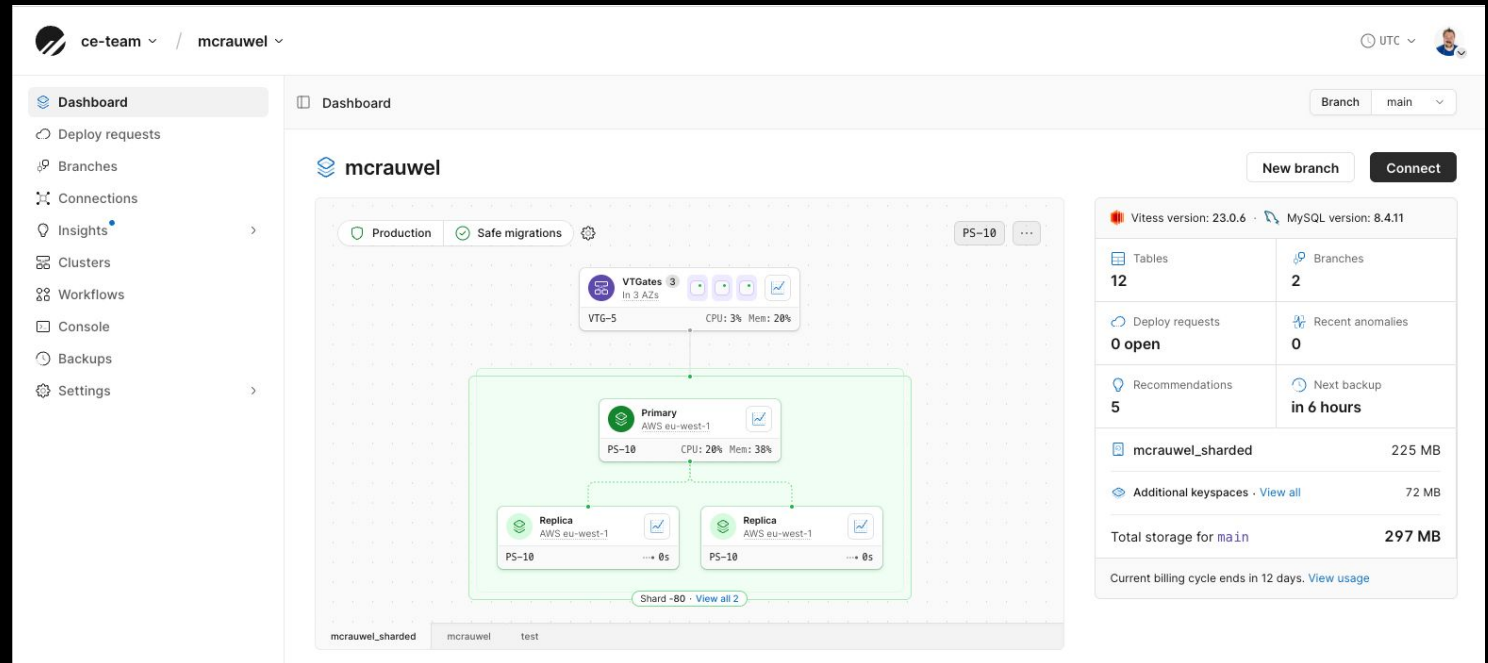
- Vitess component based on Orchestrator
- Stateless, topology information is stored in the topology service
- Discovery is already taken care of by Vitess
- Refactoring will automatically update the topo
- Requirements
 - At least one replica
 - Optional: semi-sync replication



PlanetScale

Everything in this presentation is available in the Vitess Open Source version.

PlanetScale offers a managed service Vitess if you don't want to manage this yourself.



PlanetScale - Insights



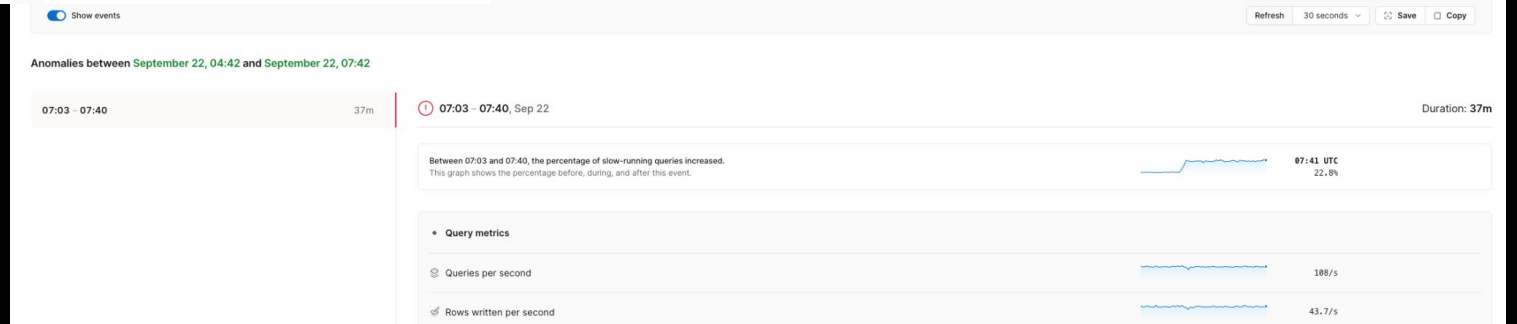
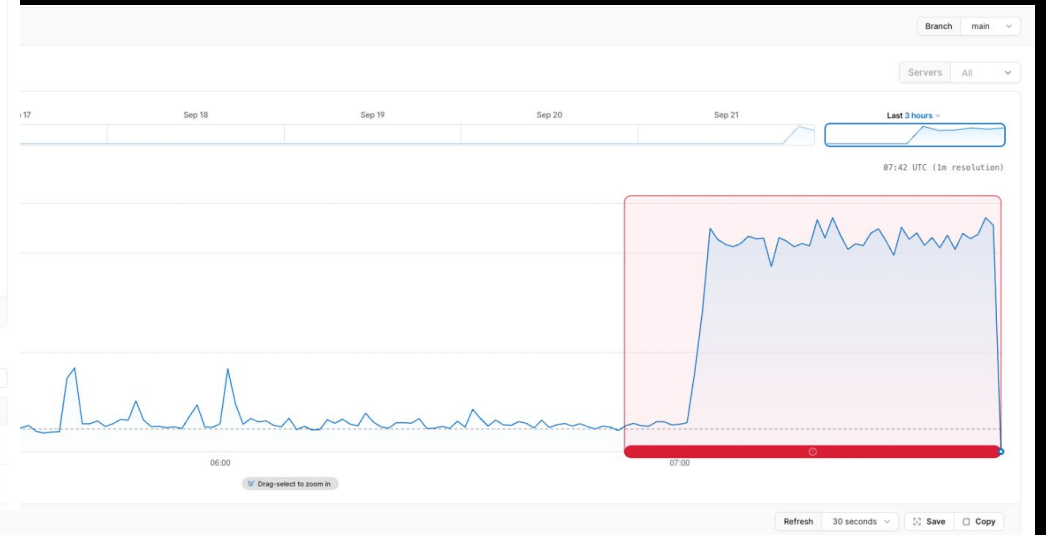
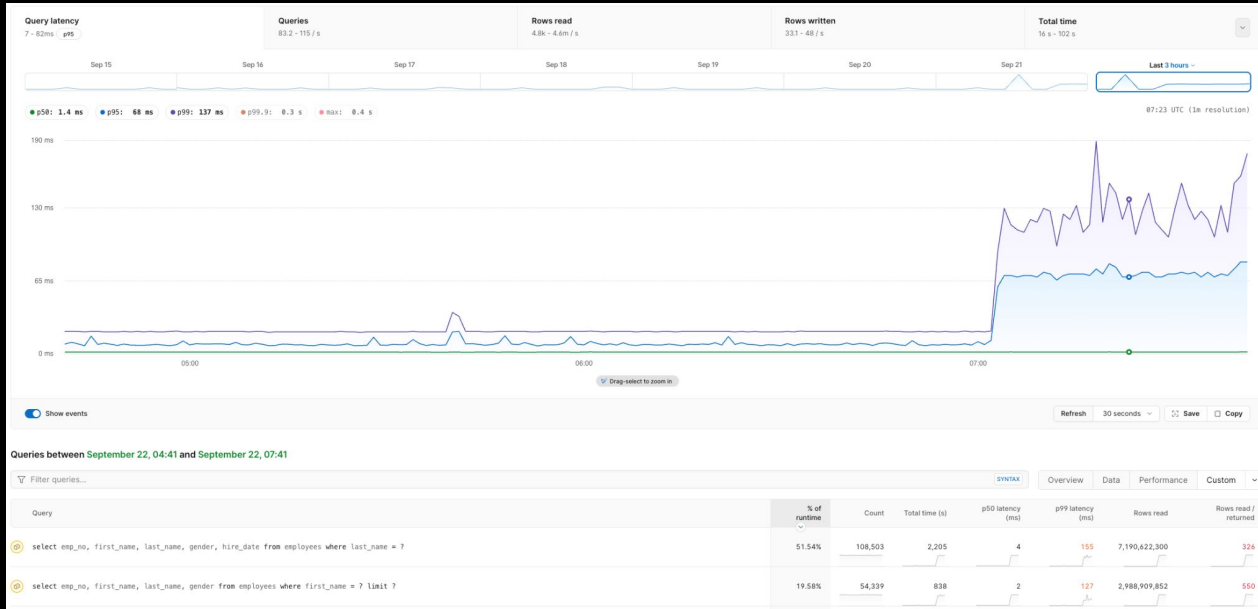
Queries between September 21, 19:00 and September 22, 07:00

Filter queries...

Query	% of runtime	Count	Total time (s)	p50 latency (ms)	p99 latency (ms)	Rows read	Rows read / returned
<code>select emp_no, first_name, last_name, gender, hire_date from employees where last_name = ?</code>	22.75%	432,025	1,532	3	6	175,415,889	2
<code>select e.emp_no, e.first_name, e.last_name, e.gender, e.hire_date, s.salary from employees as e join salaries as s on s.emp_no = e.emp_no where e.emp_no = ? and s.to_date = ?</code>	13.48%	648,554	908	1	3	4,363,226	10
<code>delete from employees where emp_no = ?</code>	9.29%	107,690	626	6	10	107,690	0
<code>insert into employees(birth_date, emp_no, first_name, gender, hire_date, last_name) values <values></code>	7.13%	107,692	480	4	8	0	0
<code>select emp_no, first_name, last_name, gender from employees where first_name = ? limit ?</code>	7.11%	215,482	479	2	5	43,092,513	2
<code>select title, from_date, to_date from titles where emp_no = ? order by from_date asc</code>	4.21%	215,829	284	1	3	251,573	1
<code>select d.dept_name, de.from_date, de.to_date from dept_emp as de join departments as d on d.dept_no = de.dept_no where de.emp_no = ? order by de.from_date asc</code>	3.55%	171,626	239	1	3	317,518	2



PlanetScale - Insights - Anomalies



PlanetScale - Deploy Requests

Keyspace: mcrauwel_sharded (2 shards)

Filter tables...

Show Changes only

Sort by Name

employees

CREATE TABLE `employees` (
 `emp\_no` int NOT NULL,
 `birth\_date` date NOT NULL,
 `first\_name` varchar(34) NOT NULL,
 `last\_name` varchar(36) NOT NULL,
 `gender` enum('M', 'F') NOT NULL,
 `hire\_date` date NOT NULL,
 PRIMARY KEY (`emp\_no`),
 + KEY `last\_name\_idx` (`last\_name`),
 + KEY `first\_name\_idx` (`first\_name`)
) ENGINE=InnoDB,
CHARSET=utf8mb4,
COLLATE=utf8mb4_0900_ai_ci;

Connect

This is a development branch

Safe migrations not enabled

Vectors not enabled

Safe migrations can be enabled if you'd like to use this development branch as a staging database.

Learn more

Branch ID: a4f8en1f8c9

0 Created tables

1 Altered table

0 Dropped tables

Deploy branch changes

Changes can be deployed to branches with safe migrations enabled.

Deploy to main

Optional information about your deploy request

Create deploy request

Deploy requests > Deploy request #27 Open Review changes

Summary Schema changes

Matthias Crauwels opened this deploy request on add-indexes to deploy to main just now

Leave a comment

Comment

employees

ALTER TABLE (2 shards) ↗

Not started

SQL Shards (0 of 2 completed)

-- The following SQL will run for this change
ALTER TABLE `employees`
ADD KEY `last\_name\_idx` (`last\_name`),
ADD KEY `first\_name\_idx` (`first\_name`)

Deploy changes

Auto-apply changes

0 Created tables

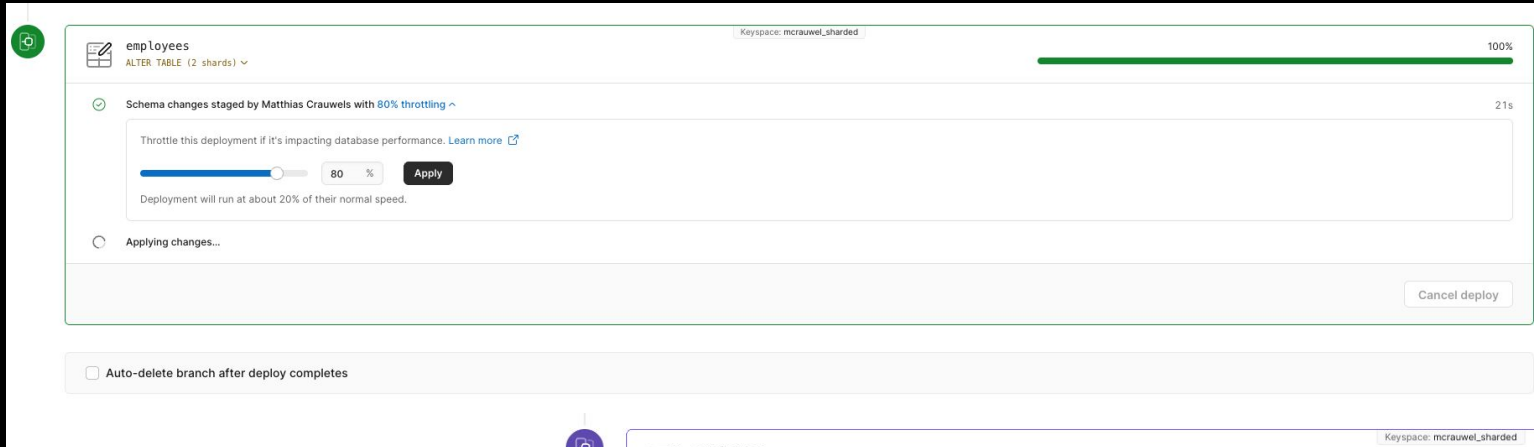
1 Altered table

0 Dropped tables

View schema changes

Close deploy request

PlanetScale - Deploy Requests



The screenshot shows the PlanetScale deployment interface for a table named 'employees' in the 'mcrauwel_sharded' keyspace. The deployment is in progress, indicated by a green progress bar at 100%. The interface shows a 'Schema changes staged by Matthias Crauwels with 80% throttling' step, which is completed. Below this, there is a section for 'Applying changes...' with a 'Cancel deploy' button. At the bottom, there is a checkbox for 'Auto-delete branch after deploy completes'.

employees
ALTER TABLE (2 shards) v

Keyspace: mcrauwel_sharded

100%

Schema changes staged by Matthias Crauwels with 80% throttling 21s

Throttle this deployment if it's impacting database performance. [Learn more](#)

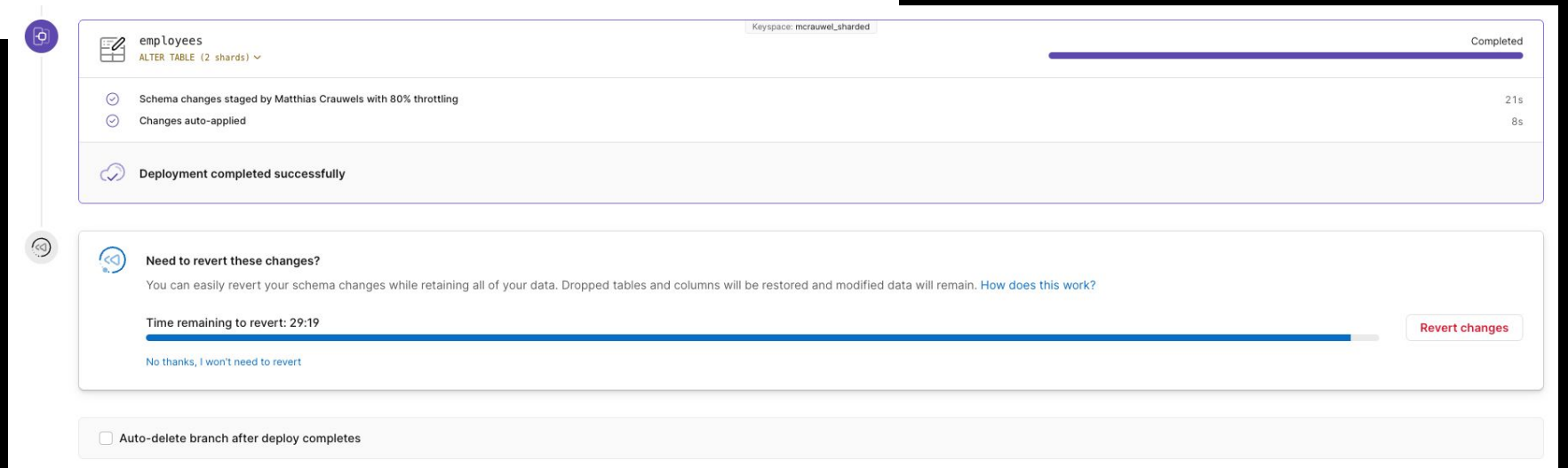
80 % Apply

Deployment will run at about 20% of their normal speed.

Applying changes...

Cancel deploy

☐ Auto-delete branch after deploy completes



The screenshot shows the PlanetScale deployment interface after the deployment has completed successfully. The progress bar is now blue and labeled 'Completed'. The interface shows a list of steps: 'Schema changes staged by Matthias Crauwels with 80% throttling' (21s), 'Changes auto-applied' (8s), and 'Deployment completed successfully'. Below this, there is a section for 'Need to revert these changes?' with a 'Revert changes' button. At the bottom, there is a checkbox for 'Auto-delete branch after deploy completes'.

employees
ALTER TABLE (2 shards) v

Keyspace: mcrauwel_sharded

Completed

Schema changes staged by Matthias Crauwels with 80% throttling 21s

Changes auto-applied 8s

Deployment completed successfully

Need to revert these changes?

You can easily revert your schema changes while retaining all of your data. Dropped tables and columns will be restored and modified data will remain. [How does this work?](#)

Time remaining to revert: 29:19

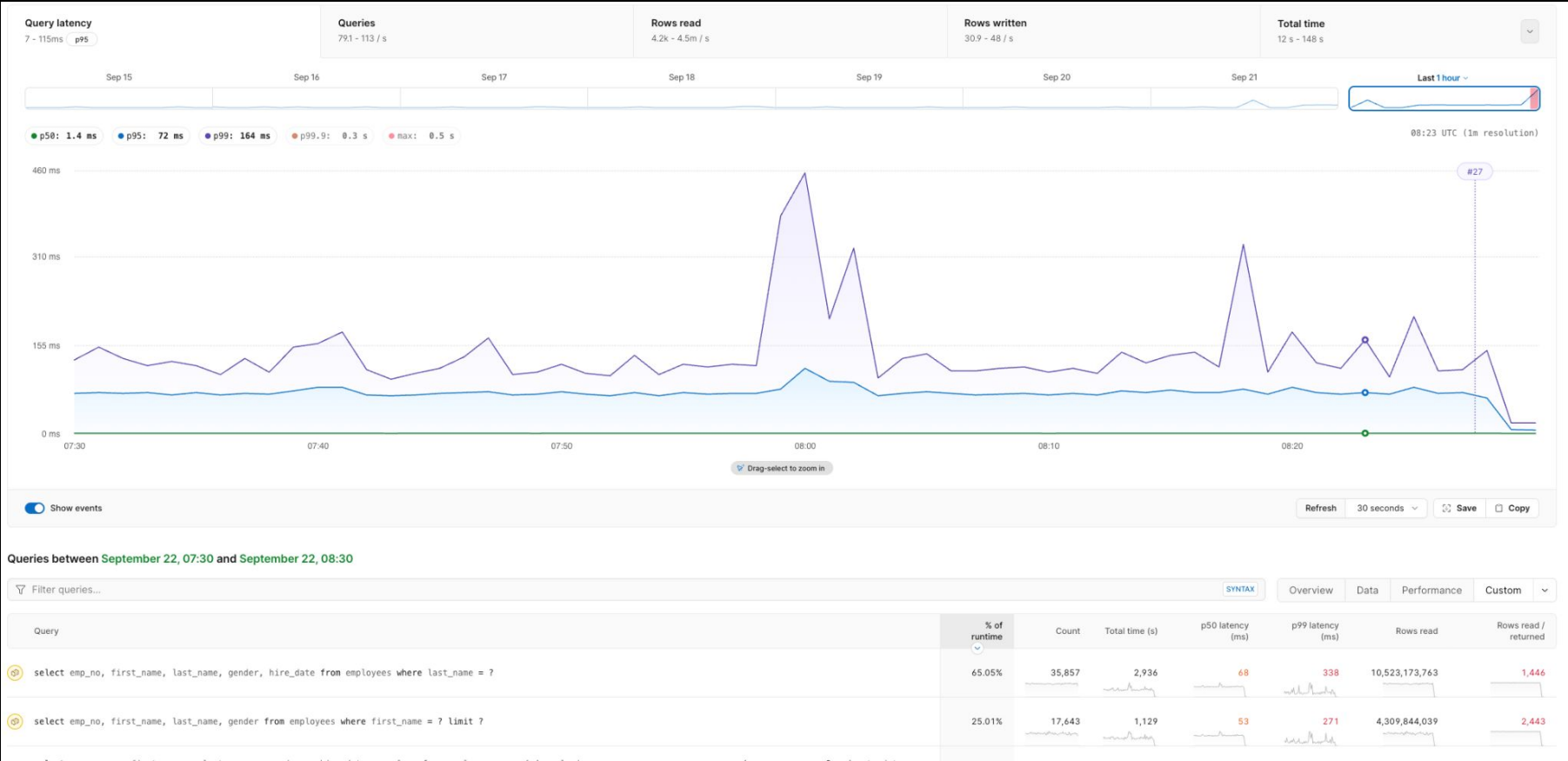
Revert changes

No thanks, I won't need to revert

☐ Auto-delete branch after deploy completes



PlanetScale - Deploy Requests



Questions?



Thank you!

